

# TOAST: Stochastic Robot Action Tokenization for Autoregressive Vision-Language-Action Models

Keisuke Shirai, Tomohiro Motoda, Hanbit Oh, Ryoichi Nakajo,  
Roman Mykhailishyn, Ryo Hanai, Shotaro Miwa, Yukiyasu Domae  
AIST

**Abstract**—Autoregressive Vision-Language-Action models often represent continuous robot actions as discrete token sequences, enabling action prediction with standard next-token objectives. FAST has substantially improved this representation by compactly encoding action containing diverse temporal frequencies into relatively few tokens. However, while such compression reduces the number of action tokens required for autoregressive prediction, it does not necessarily improve the efficiency of policy learning from limited demonstrations. In particular, FAST typically assigns a single deterministic tokenization to each quantized action sequence, although multiple token sequences can represent and decode to the same robot motion. We investigate whether exploiting this representational redundancy can improve policy learning. In this paper, we propose **Tokenization of Action Sequences with STochastic Sampling (TOAST)**, a stochastic action tokenization method that samples alternative tokenizations of the same quantized action sequence during policy training. This diversifies the discrete supervision while preserving the underlying robot action and requires no additional demonstrations. Experiments on LIBERO show that TOAST consistently improves over its deterministic counterpart, with the improvement increasing as training data decreases, achieving a 6.8 point gain in success rate when only 1/16 of training data is available. Across four real-robot manipulation tasks, TOAST further improves mean success rate by 15.8 points over the deterministic counterpart. These results demonstrate the effectiveness of stochastic action tokenization for autoregressive robot policy learning, particularly when training data are limited.

## I. INTRODUCTION

Vision-language-action models (VLAs), backed by the success of imitation learning [1], have been drawing attention as an approach for realizing general-purpose robot policies [2]–[9]. Autoregressive VLAs [2], [5], [9], [10] are policies that use a backbone multimodal large language model (MLLM) to predict action sequences as tokens via next-token prediction. Autoregressive VLAs offer advantages over flow matching-based ones [6], [7] of having a simpler architecture [11], leveraging MLLM’s pre-trained knowledge effectively [12], and realizing faster convergence [10]. In building autoregressive VLAs, the tokenizer determines what the policy has to predict, and recent work has explored efficient compression of action chunks in two ways: learned vector quantization, as in VQ-VLA [13], or training-free frequency-based compression, as in FAST [10], which applies byte pair encoding (BPE) [14] to the quantized coeffi-

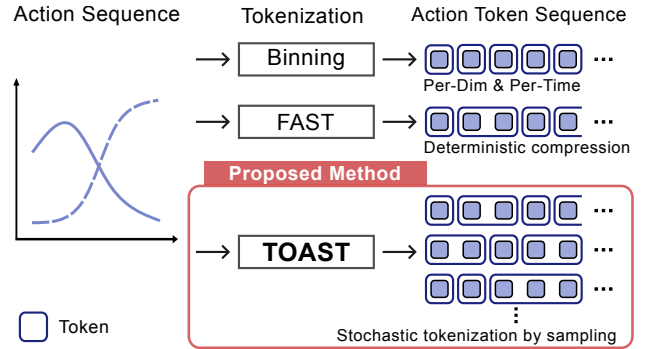


Fig. 1: We propose TOAST, a novel framework that introduces stochastic action tokenization to enable efficient policy learning with supervision diversification while preserving the underlying action.

cients of a discrete cosine transform. Both make prediction easier by shortening the token sequence.

However, the tokenization boundaries produced by frequency-based compression such as BPE are not necessarily physically meaningful for robot actions. Once the vocabulary contains tokens spanning multiple quantized action values, the same quantized action sequence can be represented by different token sequences, all of which decode to exactly the same motion. Deterministic tokenizers nevertheless select a single tokenization according to corpus-level frequency statistics, and the policy is trained under cross-entropy to reproduce only that sequence. A similar ambiguity has long been recognized in natural language processing, where stochastic tokenization [15]–[17] mitigates over-reliance on a single segmentation and has shown particular benefits when training data are limited. This issue can be especially important in robot learning, where demonstrations are costly to collect. When only limited training data are available, repeatedly supervising each action sequence with the same deterministic tokenization may encourage the policy to overfit to specific token patterns or segmentation boundaries that are not uniquely tied to the underlying motion.

To address this issue, we propose **Tokenization of Action Sequences with STochastic Sampling (TOAST)**, the first stochastic action tokenization framework for autoregressive robot policy learning. As shown in Figure 1, TOAST samples multiple tokenizations of the same action sequence during training, exposing the policy to different discrete represen-

tations while preserving the underlying motion. Moreover, because multiple valid token sequences can be obtained from each demonstration without collecting additional robot trajectories, TOAST increases the diversity of supervision available from limited data. We therefore hypothesize that stochastic action tokenization becomes increasingly beneficial when fewer demonstrations are available. We instantiate this framework with a unigram tokenizer that enables probabilistic segmentation, together with dimension-major flattening for frequency-based action sequence compression.

We evaluate this hypothesis through both controlled simulation and real-world manipulation task experiments. The results show that stochastic tokenization consistently improves policy performance over its deterministic counterpart across training-set sizes, flattening orders, and vocabulary corpora. Notably, the relative benefit of stochastic tokenization is greater when fewer demonstrations are available, indicating improved data efficiency in policy learning. We further observe substantial gains on real-robot manipulation tasks, where collecting demonstrations is particularly expensive. These results show that stochastic action tokenization provides a simple and effective means of extracting more useful supervision from limited robot demonstrations.

Our contributions are summarized as follows:

- We identify deterministic action tokenization as a source of over-reliance on a single discrete representation and formulate stochastic tokenization as a representation-level regularization effect for autoregressive VLAs.
- We introduce TOAST, which samples equivalent tokenizations of the same motion during training, yielding more diverse supervision from each demonstration without additional robot data.

## II. RELATED WORK

**Vision-language-action models and action tokenization.** Inspired by advances in foundation models for natural language processing and computer vision [18]–[22], Vision-Language-Action (VLA) models have emerged as a promising approach to generalist robot policy learning [2]–[5], [7], [23]–[25]. Autoregressive VLAs formulate action prediction as next-token prediction [2], [5], [9], [10], making the representation of continuous robot actions as discrete tokens an important design choice. Early approaches use uniform binning to discretize each action dimension and timestep independently [2], [5]. More recent methods exploit temporal or cross-dimensional structure to obtain more compact action representations. BEAST [26] represents action sequences using B-spline control points, while vector-quantization-based methods [27], [28], such as VQ-VLA [13], learn discrete latent representations of continuous actions. FAST [10] instead provides a training-free tokenizer by applying the discrete cosine transform (DCT) followed by byte pair encoding (BPE) [14]. Our work builds on this frequency-based approach, but focuses on stochastic tokenization of action sequences rather than introducing another compression scheme.

**Stochastic tokenization with frequency-based tokenizers.** In natural language processing, subword tokenizers such as BPE [14] and unigram language model [15] tokenization construct vocabularies from corpus statistics. Given a fixed vocabulary, the same underlying sequence can generally admit multiple valid segmentations, although standard tokenization selects a single one deterministically. Subword regularization [15] instead samples alternative tokenizations during training, and subsequent work has explored related stochastic tokenization strategies [16], [17], [29], [30]. These methods have been shown to enable efficient learning in low-resource settings. Frequency-based action tokenizers such as FAST [10] admit the same segmentation ambiguity, but whether sampling equivalent tokenizations improves robot policy learning has remained unexplored. This work investigates this question.

## III. PRELIMINARIES

**Vision-language-action models.** Given a language instruction  $l$  and the current robot observation  $o$ , a VLA model predicts an  $H$ -step action chunk  $a_{1:H}$ . Each action  $a_i \in \mathbb{R}^D$  consists of  $D$  action dimensions, where  $D$  depends on the robot embodiment and action representation. Autoregressive VLAs represent the action sequence as discrete tokens and model their distribution as  $\pi_\theta(t_{1:N} \mid l, o)$ . The predicted token sequence  $t_{1:N} = t_1 \dots t_N$  is decoded back into the continuous action sequence  $a_{1:H}$  for execution. The action horizon  $H$  is fixed by the policy configuration, whereas the number of tokens  $N$  depends on the tokenizer and the action sequence.

**Action sequence tokenization and compression.** Action sequence tokenization maps a continuous action chunk  $a_{1:H}$  into a one-dimensional sequence of discrete tokens  $t_{1:N}$ . The action chunk is first transformed and discretized into  $q \in \mathbb{Z}^{D \times H}$ , for example by scalar binning or by applying DCT followed by discretization. The DCT represents each dimension’s  $H$ -step trajectory by  $H$  frequency coefficients, most of which become zero after scaling and rounding, making the flattened sequence highly compressible. The resulting discrete values are then flattened into a one-dimensional sequence. Timestep-major flattening orders the values as  $q_{1,1} \dots q_{D,1} q_{1,2} \dots q_{D,H}$ , whereas dimension-major flattening orders them as  $q_{1,1} \dots q_{1,H} q_{2,1} \dots q_{D,H}$ . With scalar binning [5], each flattened discrete value directly corresponds to one action token, yielding  $N = DH$ . FAST [10] instead applies byte pair encoding (BPE) [14] to a DCT-transformed and discretized action sequence, producing a variable-length token sequence with  $N \leq DH$ .

## IV. METHOD

TOAST constructs a unigram language model over quantized action sequences (Section IV-A), samples alternative tokenizations that preserve the underlying action (Section IV-B), and uses the sampled token sequences for policy training (Section IV-C). The following sections describe each component in turn. Figure 2 shows the tokenization pipeline.

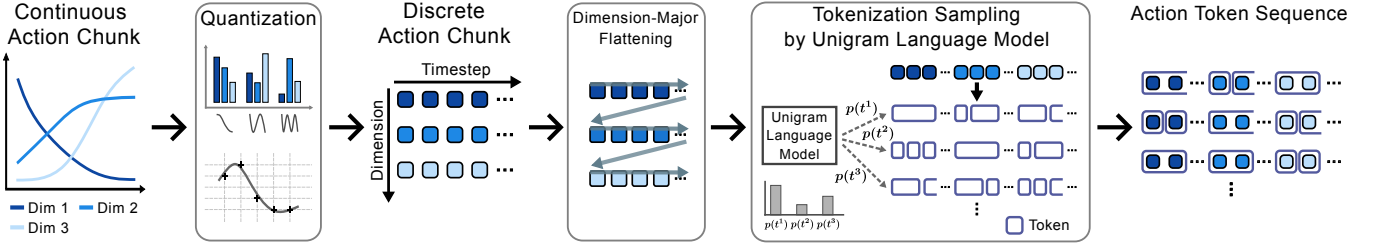


Fig. 2: **TOAST tokenization pipeline.** Given a discrete action chunk after quantization (e.g., DCT) and flattening, TOAST uses a unigram language model to calculate probabilities over the  $n$ -best token sequences for the chunk and then samples a token sequence. Tokenization sampling is performed only during training.

#### A. Constructing Action Sequence Tokenizer

To enable stochastic tokenization of action chunks, TOAST first constructs a unigram language model over quantized action sequences. We use dimension-major flattening, which places temporally adjacent values of the same action dimension consecutively in the flattened sequence. Given a corpus of quantized and flattened action sequences  $\mathcal{Q} = \{q^1, \dots, q^M\}$ , defined in Section III, we estimate a vocabulary  $\mathcal{V}$  whose tokens correspond to either individual quantized action values or contiguous subsequences of values. Following subword regularization [15], we first construct a large seed vocabulary from candidate subsequences observed in  $\mathcal{Q}$ , while retaining all individual quantized values to ensure that any action sequence can be represented. The unigram model assigns each token  $v \in \mathcal{V}$  a probability  $p(v)$ . The vocabulary and token probabilities are then iteratively optimized using the standard unigram language-model procedure, and low-contribution tokens are pruned until the target vocabulary size is reached. Given the resulting unigram language model, a flattened action sequence  $q$  can generally be tokenized into multiple valid token sequences  $t$ . The probability of a token sequence is computed as  $P(t) = \prod_{i=1}^N p(t_i)$ , where  $t_i$  denotes the  $i$ -th token in  $t$ . The unigram language model thereby assigns probabilities to alternative tokenizations of the same action sequence.

#### B. Stochastic Tokenization of Action Chunks

The unigram language model constructed in Section IV-A allows a quantized and flattened action sequence ( $q$ ) to be represented by multiple token sequences with different segmentation boundaries. While deterministic tokenization selects a single token sequence from these possible representations, TOAST probabilistically samples one of them. Given  $q$ , we obtain  $n$  candidate token sequences  $\{t^1, t^2, \dots, t^n\}$  by  $n$ -best search via unigram language model scores. Following subword regularization [15],  $n$ -best search is performed in linear time by Forward-DP Backward-A\* algorithm [31], and the probability of the  $i$ -th candidate tokenization is approximated as

$$P(t^i | q) \simeq \frac{P(t^i)^\alpha}{\sum_{j=1}^n P(t^j)^\alpha}, \quad (1)$$

where  $\alpha$  controls the sharpness of the multinomial distribution. A larger  $\alpha$  leads to deterministic tokenization,

while a smaller  $\alpha$  leads to uniform sampling. Importantly, all sampled token sequences decode to the same quantized action sequence  $q$ . Thus, TOAST diversifies its discrete representation while preserving the underlying robot action.

#### C. Policy Training with TOAST

In policy training with TOAST, a token sequence  $\tilde{t} = \tilde{t}_1 \tilde{t}_2 \dots \tilde{t}_{\tilde{N}}$  is sampled and used as the target at each training step. The policy is optimized to minimize the following standard next-token prediction objective:

$$-\sum_{i=1}^{\tilde{N}} \log \pi(\tilde{t}_i | l, o, \tilde{t}_{<i}). \quad (2)$$

Thus, TOAST leaves the policy architecture and learning objective unchanged and modifies only the discrete representation used as the training target. At inference time, tokenization sampling is not applied. The policy autoregressively predicts an action token sequence, which is decoded into the corresponding continuous action sequence in the same manner as conventional token-based autoregressive policies.

### V. EXPERIMENTS

We evaluate TOAST by addressing the following research questions: **(Q1)** Does stochastic tokenization improve policy learning, particularly when training data is limited? **(Q2)** Does stochastic tokenization remain effective in real-world robot manipulation? **(Q3)** How sensitive is TOAST to tokenizer design choices? The remainder of this section explains experimental setup in Section V-A and answers **Q1**, **Q2**, and **Q3** in Section V-B, Section V-C, and Section V-D, respectively.

#### A. Experimental Setup

**Evaluation tasks.** Evaluation is performed on both simulation and real-robot tasks (Figure 3). We chose the **LIBERO** benchmark [32] for simulation experiments. The benchmark features Franka Emika Panda with the control frequency of 10 Hz. Actions are 7-dimensional values of a 6-DoF end-effector pose and an absolute gripper pose. LIBERO consists of four task suites: LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-Long [32].

The real-robot experiments feature Franka Research 3 with the control frequency of 20 Hz, and we designed the following four manipulation tasks:

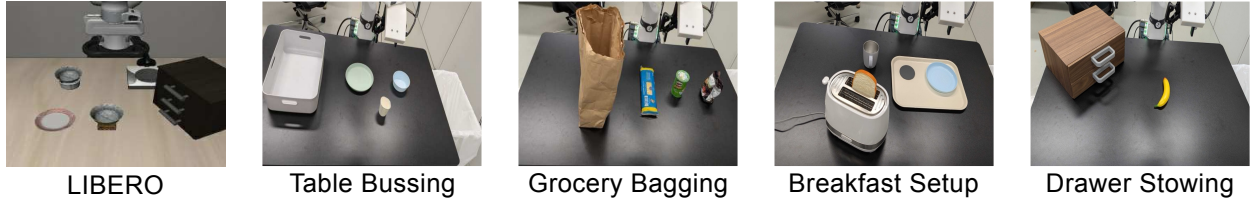


Fig. 3: **Evaluation tasks.** We evaluate TOAST on the LIBERO benchmark and four real-robot manipulation tasks.

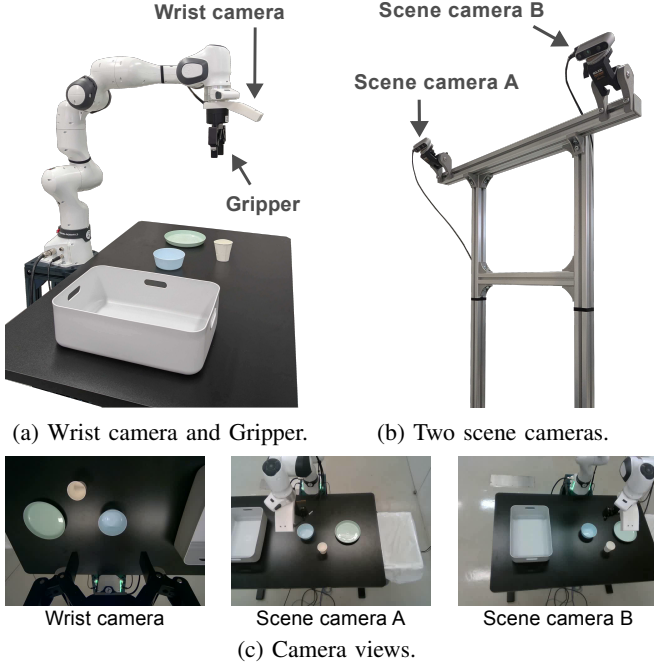


Fig. 4: **Our real robot setup.** The robot (Franka Research 3) is equipped with a wrist camera (RealSense D405) and a Robotiq 2F-85 Gripper. Two scene cameras (RealSense D435) are placed facing the robot.

- **Table Bussing** sorts trash on the desk into the trash bin and dishes into the dish rack, requiring accurate grasping of objects.
- **Grocery Bagging** puts grocery items into a paper bag, requiring careful insertion of objects.
- **Breakfast Setup** moves a slice of toast onto a plate and a cup onto a coaster, requiring precise placement of objects at the intended locations.
- **Drawer Stowing** opens a drawer, puts an object inside, and closes it, requiring precise interaction with an articulated object.

The actions are represented as 8-dimensional values of 7-DoF joint angles and an absolute gripper pose. As shown in Figure 4, the robot was equipped with a single RealSense D405 as a wrist camera, two RealSense D435 as scene cameras, and a Robotiq 2F-85 gripper.

Table I shows the number of demonstrations collected for each task. We collected demonstrations by teleoperation with GELLO [33]: 200 for Table Bussing, 200 for Grocery Bagging, 100 for Breakfast Setup, and 60 for Drawer Stowing.

TABLE I: Amount of real-robot task demonstrations used for training policies.

| Table Bussing | Grocery Bagging | Breakfast Setup | Drawer Stowing |
|---------------|-----------------|-----------------|----------------|
| 200           | 200             | 100             | 60             |

TABLE II: Average episode duration of the datasets (in seconds).

| LIBERO | Table Bussing | Grocery Bagging | Breakfast Setup | Drawer Stowing |
|--------|---------------|-----------------|-----------------|----------------|
| 16.15  | 38.47         | 41.00           | 41.37           | 39.48          |

The number of demonstrations scales with the number of objects that the robot picks up in a task. Table II shows average episode durations of the evaluation tasks.

**Policy configuration.** All policies use PaliGemma-3B [20] as the backbone MLLM and are fine-tuned from vision-language-pretrained weights without prior action-policy training. This avoids biasing the comparison toward a particular action tokenizer through action-pretrained initialization. We implement all policies within the same autoregressive VLA pipeline based on OpenPI<sup>1</sup>. Actions are represented relative to the current proprioceptive state [34], except for the gripper state, which is represented in absolute coordinates. For each data scale, all methods are trained on identical demonstration subsets and use the same policy architecture and optimization settings. All policies are trained for 30,000 steps with a batch size of 32 using AdamW optimizer [35]. The action chunk horizon is set to 10 for LIBERO and 20 for the real-robot tasks to predict 1-second action chunks. Action values are normalized to  $[-1, 1]$ , with the bounds set to the 1st and 99th percentiles of each action dimension within each dataset. All results are produced in our pipeline rather than taken from prior papers.

**Tokenizer configuration.** We compare TOAST with Binning [5], FAST+ and FAST (rebuilt) [10], VQ-VLA [13], and BEAST [26]. We use VQ-VLA as the representative learned tokenizer because it is a residual VQ tokenizer with a public reference implementation. Unless otherwise specified, TOAST uses DCT quantization with a scaling factor of 10, dimension-major flattening, and a unigram vocabulary of 512 action tokens. The unigram tokenizer is implemented with SentencePiece [36]. For stochastic tokenization, we use  $n = 64$  candidate token sequences with  $\alpha = 0.1$  on LIBERO

<sup>1</sup><https://github.com/Physical-Intelligence/openpi>

and  $\alpha = 0.5$  on the real robot.<sup>2</sup> TOAST (deterministic) uses the same quantization, flattening, vocabulary, and policy configuration, but deterministically selects a single tokenization, thereby isolating the effect of stochastic tokenization.

Following FAST [10], which builds its vocabulary on a large external corpus, we construct the TOAST vocabulary from 1M DROID [37] action chunks (end-effector actions for LIBERO and joint angles for the real robot). An external corpus keeps the tokenizer fixed across data scales, so that only the policy training data varies in Section V-B. Vocabularies built from the target demonstrations are examined in Section V-D. FAST+ is the publicly released FAST tokenizer<sup>3</sup> and uses DCT quantization and BPE compression, with 2048-token vocabulary built on 1M action chunks across multiple embodiments [10]. For fair comparison, we also use FAST (rebuilt), which uses 512-token vocabulary rebuilt from the same DROID corpus as TOAST. Binning discretizes each normalized action value in  $[-1, 1]$  into 256 uniform bins, yielding 70 tokens per  $10 \times 7$  action chunk, and is fixed and data-independent. VQ-VLA uses the reference residual vector-quantized tokenizer with four 256-code quantizers over non-overlapping 5-step windows, yielding 8 tokens per action chunk. Because a DROID-only VQ-VLA tokenizer produced poor LIBERO reconstruction and 0% downstream success rate, we train its codebook on DROID together with the corresponding LIBERO training data and refit it at each data scale to avoid using held-out trajectories. BEAST follows its LIBERO configuration with degree-4 B-splines, five basis functions, and 256-bin quantization, yielding 35 tokens per chunk. It requires no learned vocabulary and is therefore fixed across data scales. We use a fixed quantization range of  $[-2, 2]$  instead of the batch-statistics-based range in the reference implementation.

**Evaluation protocol.** On LIBERO, we follow the evaluation protocol of prior work [5], evaluating 500 rollouts per suite (50 per task) for each random seed, totaling 2,000 rollouts. For the real-robot experiments, we evaluate 30 rollouts per task for each random seed, with 5 rollouts for each of 6 task configurations varying object poses, colors, or locations. The same evaluation configurations are used across all methods. We report the mean success rate across three independently trained policies with different random seeds in both settings. Significance is assessed with a hierarchical bootstrap that resamples seeds as clusters and episodes within each seed (10,000 resamples), pairing methods on identical evaluation episodes. A difference is marked significant when its two-sided 95% percentile CI excludes zero.

### B. Simulation Evaluation across Data Scales

We first aim to address **Q1** through controlled experiments on LIBERO. To examine the performance of the policy under different amounts of training data, we train policies using

<sup>2</sup> $\alpha = 0.5$  for the real robot was chosen in preliminary trials. As shown in Section V-D, both values lie on the plateau, we do not expect this difference to affect the conclusions.

<sup>3</sup><https://huggingface.co/physical-intelligence/fast>

TABLE III: **LIBERO Results with different amounts of training data.** Mean success rate is calculated based on 2,000 rollouts per seed. \* denotes that TOAST is significantly better than this row (the 95% bootstrap CI of the difference excludes zero). This shows that stochastic action tokenization is more effective as available training data is reduced.

| Tokenizer             | Fraction of LIBERO training data |             |             |             |             |
|-----------------------|----------------------------------|-------------|-------------|-------------|-------------|
|                       | 1/1                              | 1/2         | 1/4         | 1/8         | 1/16        |
| Binning [5]           | 89.5                             | 80.0*       | 70.7*       | 56.4*       | 37.8*       |
| FAST+ [10]            | 85.7*                            | 59.4*       | 40.6*       | 30.1*       | 17.8*       |
| FAST (rebuilt) [10]   | <b>92.4</b>                      | 83.8        | 71.3*       | 55.7*       | 34.4*       |
| BEAST [26]            | 91.5                             | 82.6        | 69.8*       | 55.2*       | 37.1*       |
| VQ-VLA [13]           | 82.8*                            | 74.7*       | 58.3*       | 46.8*       | 31.6*       |
| TOAST (deterministic) | 90.9                             | 82.7*       | 69.8*       | 55.0*       | 36.0*       |
| TOAST                 | 91.4                             | <b>84.9</b> | <b>75.3</b> | <b>60.2</b> | <b>42.8</b> |

$1/k$  ( $k \in \{1, 2, 4, 8, 16\}$ ) of the available LIBERO demonstrations. For each data scale, subsets are drawn uniformly at random per suite with a fixed seed, and identical subsets are used across all methods. The evaluation protocol is kept fixed regardless of the data scale.

Table III shows the results. TOAST improves over its deterministic counterpart at every data scale, and the gain widens as training data is reduced, and it is significant with half or less of the data. Among the other deterministic baselines, the policies trained with FAST (rebuilt) and BEAST achieve performance comparable to that of the policy trained with TOAST when sufficient training data are available. As the amount of training data decreases, however, the performance gap between these baselines and TOAST becomes larger. With one quarter or less of the demonstrations, TOAST achieves the highest performance among the evaluated methods. Since FAST (rebuilt) uses the same quantization, tokenizer-construction data, and vocabulary size as TOAST, these results suggest that the improvement in the low-data settings is not due only to these factors, but also to stochastic tokenization. These results answer **Q1** affirmatively: stochastic tokenization improves policy learning, and its benefit grows as demonstrations become scarcer.

### C. Real-Robot Evaluation

We next address **Q2** through real-robot experiments. The primary comparison is between TOAST and its deterministic counterpart, which isolates the effect of stochastic action tokenization under identical configurations. We additionally include FAST+ as an external reference because it is a publicly released deterministic tokenizer based on the same frequency-based compression paradigm as TOAST. FAST (rebuilt) differs from TOAST (deterministic) only in BPE vs. unigram segmentation and the flattening direction, and the two are within 1.6 points at every data scale in Table III. We therefore regard TOAST (deterministic) as a proxy for FAST (rebuilt) on the real robot. The policy decodes tokens greedily and maps them back to a  $D \times H$  chunk by vocabulary lookup and inverse DCT; on the real robot, generating a 1-second (20-step) joint-angle chunk takes 168 ms on a single RTX

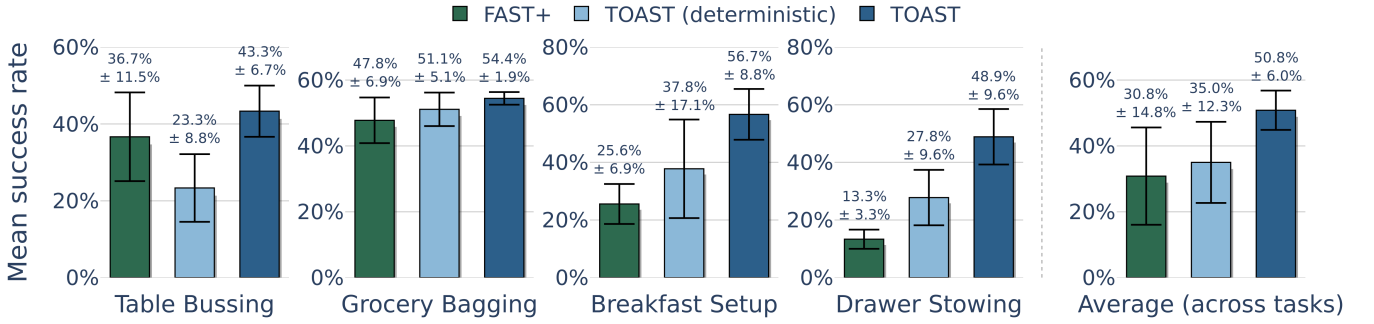


Fig. 5: **Real-robot task results.** We evaluate policies with three different tokenizations: TOAST, TOAST (deterministic), and FAST+. Each policy is evaluated based on 30 rollouts per task with three random seeds. TOAST achieves the highest success rate (50.8%), improving performance of TOAST (deterministic) by 15.8 points and FAST+ by 20.0 points.

4090. Chunks are executed open-loop: the robot tracks the 20 Hz targets with a 1 kHz joint impedance controller using zero-order hold, and keeps executing the current chunk while the next one is generated. If a predicted sequence cannot be decoded into a complete chunk, it is discarded and inference is re-run on the next observation. This rarely occurs, and the same pipeline is applied identically to all tokenizers.

Figure 5 shows the results on the real-robot tasks. We observe that TOAST outperforms TOAST (deterministic) by 15.8 points and FAST+ by 20.0 points, and both differences are significant under the same bootstrap protocol as in Section V-B. While stochastic tokenization yields larger improvements in the real-robot experiments than in the full-data simulation setting, this result is consistent with the data-scaling trend observed on LIBERO, where the benefit of sampling increases as the amount of training data decreases. Since real-robot demonstrations are substantially more limited than those in simulation, these results further support the practical value of stochastic action tokenization in data-constrained real-world scenarios. Since the simulation and real-robot evaluations also differ in task distribution, action representation, and execution conditions, we do not attribute the larger real-robot gains to data scale alone.

The results also show that larger gains from stochastic tokenization are obtained in all tasks except Grocery Bagging. During the rollouts, we observed that the deterministic baselines frequently fail on actions that require higher precision, such as grasping the rim of a plate or accurately reaching a drawer handle. These errors often lead directly to task failure, whereas the policy trained with stochastic tokenization executes such interactions more reliably. This observation suggests that the larger real-robot gains are associated in part with improved robustness in action segments that require precise manipulation.

#### D. Analysis of Tokenization Design Choices

We next address **Q3** by analyzing the design and behavior of stochastic action tokenization. Specifically, we examine the sensitivity to the sampling parameter  $\alpha$ , the corpus used to construct the vocabulary, the flattening order, and the policy’s sensitivity to equivalent tokenizations, and finally report the training overhead of sampling.

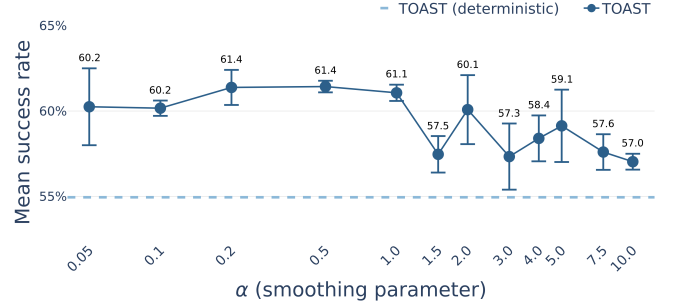


Fig. 6: **LIBERO (1/8) results with varying sampling hyperparameter  $\alpha$ .** Performance is stable for  $\alpha \in [0.05, 1.0]$  and degrades toward the deterministic baseline for larger  $\alpha$ .

**Sensitivity to the sampling hyperparameter  $\alpha$ .** We first examine the sensitivity of stochastic tokenization to the smoothing parameter  $\alpha$ , which controls the concentration of the sampling distribution over alternative tokenizations. We conduct this analysis on LIBERO using 1/8 of the training demonstrations, where the effect of stochastic tokenization is pronounced. As shown in Figure 6, policy performance remains stable over a broad range of  $\alpha$  from 0.05 to 1.0 and consistently exceeds the deterministic counterpart. Increasing  $\alpha$  concentrates the sampling distribution toward high-probability tokenizations, approaching deterministic tokenization, as in subword regularization [15]. Consistent with this behavior, policy performance generally decreases at larger  $\alpha$ , although the trend is not strictly monotonic.

**Effect of the vocabulary corpus.** Our default tokenizer vocabulary is built from DROID, an external corpus that is much larger than the policy training data. To examine whether the benefit of stochastic tokenization depends on this external corpus, we construct vocabularies from DROID+LIBERO and from LIBERO only, where the LIBERO portion is restricted to the same  $1/k$  subset used for policy training, and keep all other settings unchanged. Table IV shows the results. With both vocabularies, TOAST significantly outperforms its deterministic counterpart at every data scale, including full data. Notably, the deterministic counterpart barely changes across vocabularies (e.g., 36.0,

TABLE IV: **Effect of the vocabulary corpus.** LIBERO success rate of TOAST and TOAST (deterministic) with vocabularies built from DROID+LIBERO and only LIBERO. The LIBERO portion is restricted to the same  $1/k$  subset used for policy training. \*: TOAST is significantly better than TOAST (deterministic).

| Tokenizer                           | Fraction of LIBERO training data |       |       |       |       |
|-------------------------------------|----------------------------------|-------|-------|-------|-------|
|                                     | 1/1                              | 1/2   | 1/4   | 1/8   | 1/16  |
| <i>(Vocabulary: DROID + LIBERO)</i> |                                  |       |       |       |       |
| TOAST (deterministic)               | 90.1*                            | 81.8* | 69.2* | 53.0* | 36.4* |
| TOAST                               | 92.2                             | 85.3  | 74.7  | 61.3  | 46.0  |
| <i>(Vocabulary: LIBERO)</i>         |                                  |       |       |       |       |
| TOAST (deterministic)               | 89.1*                            | 80.3* | 67.5* | 53.8* | 37.9* |
| TOAST                               | 91.3                             | 83.8  | 73.2  | 65.0  | 49.8  |

36.4, and 37.9 at 1/16 for DROID, DROID+LIBERO, and LIBERO only), whereas the gain from sampling grows as the vocabulary is built closer to the target domain (+6.8, +9.6, and +11.9, respectively). We attribute this to the vocabulary containing more tokens that match the target action patterns, which yields more plausible alternative segmentations for sampling to exploit. We keep the DROID vocabulary as the default to keep the tokenizer independent of the policy training data and consistent with the FAST (rebuilt) comparison. The results here indicate that this is a conservative choice. Thus, the low-data benefit of stochastic tokenization does not rely on an external corpus and persists, and in fact grows, even when the vocabulary is constructed solely from the target demonstrations.

**Effect of the flattening direction.** Our default dimension-major flattening places temporally adjacent values of the same action dimension consecutively, whereas timestep-major flattening in previous work [5], [10] arranges all action dimensions at each timestep consecutively. We examine the impact of stochastic tokenization on different flattening directions using LIBERO. Concretely, we compare performance improvements by sampling (success rate gain of TOAST from TOAST (deterministic)) across different amounts of LIBERO training data in both flattening directions.

Figure 7 shows success rate gains by sampling in both flattening directions. Dimension-major flattening tends to achieve higher performance improvement than timestep-major flattening in our experiments, and it achieves 6.8 points higher absolute performance at 1/16 scale (42.8% by dimension vs 36.0% by timestep), supporting its use as the default configuration, while the two flattenings are on par (91.4 vs. 91.3) at full data. The larger gain of timestep-major flattening at 1/8 (+8.6) reflects its lower deterministic baseline (48.0 vs. 55.0) rather than a higher absolute performance (56.6 vs. 60.2). More importantly, however, the benefit of stochastic tokenization is not specific to this flattening choice. Sampling improves performance with timestep-major flattening, with clear gains when fewer training data are available. Thus, although the flattening order affects absolute performance, sampling improves both representations with gains that grow as data shrink (significant at 1/4 and below in

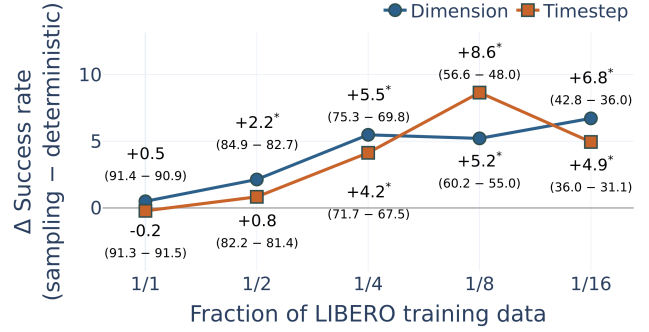


Fig. 7: **Success rate gain by sampling.** \* denotes a significant improvement over the deterministic baseline. In both flattening directions, stochastic action tokenization tends to achieve more improvements when limited data is available.

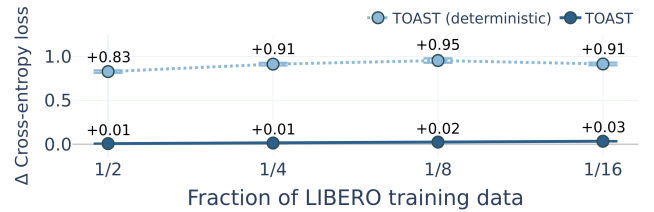


Fig. 8: **Sensitivity to equivalent tokenizations** ( $\Delta_{\text{tok}} = L_{\text{alt}} - L_{\text{can}}$ ) on 100 held-out LIBERO demonstrations. Stochastic tokenization reduces dependence on a single tokenization of an action chunk.

both directions), indicating that the low-data benefit observed in Section V-B is not an artifact of the flattening order used by TOAST.

**Sensitivity to equivalent action tokenizations.** We examine whether stochastic tokenization reduces the policy’s dependence on a particular tokenization of an action sequence. A frequency-based tokenizer can represent the same quantized action sequence using multiple valid token sequences, although deterministic training exposes the policy to only one of them. For each held-out LIBERO action chunk, we compute the cross-entropy loss for its canonical tokenization and for 63 alternative tokenizations sampled from the same unigram model. Since all of these tokenizations decode to the same quantized action sequence, their loss difference measures sensitivity to the choice of discrete representation. We define this quantity as  $\Delta_{\text{tok}} = L_{\text{alt}} - L_{\text{can}}$ , where  $L_{\text{alt}}$  is averaged over the alternative tokenizations. We use this metric as a representation-level diagnostic, rather than as a proxy for closed-loop policy performance.

As shown in Figure 8, deterministic training produces a consistently large positive loss gap across data scales, indicating a strong preference for the canonical tokenization used during training. In contrast, stochastic training reduces this gap to nearly zero, showing less sensitivity to how the same underlying action is segmented into tokens. This is not because stochastic training inflates the loss overall. On held-out data, TOAST attains lower  $L_{\text{can}}$  than its deterministic

counterpart (1.146 vs. 1.252 at 1/8), indicating better generalization. We observe the same qualitative behavior on the training trajectories, suggesting that this difference reflects the representation learned by the policy rather than a held-out generalization effect. These results confirm that stochastic training has the intended effect of reducing dependence on a single tokenization of an action sequence.

**Computational cost.** Averaged over 1,000 training steps on each of the five tasks, one step takes 0.417 s with TOAST, 0.417 s with TOAST (deterministic), and 0.415 s with FAST+ (0.4% increase). This demonstrates that  $n$ -best search runs in linear time and its sampling cost is negligible.

## VI. LIMITATIONS

First, TOAST relies on multiple valid segmentations of the same quantized sequence and therefore does not directly apply to learned vector-quantized tokenizers [13], [28], [38]. Extending stochastic tokenization to learned discrete representations is left for future work. Second, our experiments use a single Franka embodiment with end-effector actions in simulation and joint-angle actions on the real robot. We do not study a shared vocabulary across embodiments as in [10], [28]. Third, we instantiate stochastic tokenization with a unigram model and subword regularization [15]. Other stochastic tokenization schemes such as BPE-dropout [16] were not compared, and whether the benefit stems from the unigram model specifically or from stochastic segmentation in general remains open. Finally, the vocabulary size (512) and the number of candidates  $n$  were fixed throughout, and their sensitivity is not examined.

## VII. CONCLUSION

We presented TOAST, a stochastic action sequence tokenization method for autoregressive robot policies. By sampling multiple tokenizations that represent the same quantized action sequence during training, TOAST provides diverse discrete supervision without additional robot demonstrations. Experiments on LIBERO show that stochastic tokenization consistently improves over its deterministic counterpart, with larger gains when fewer demonstrations are available. We observe consistent improvements across diverse real-robot manipulation tasks. Analyses show that these gains are robust to the flattening order, sampling parameter, and vocabulary corpus, while stochastic training substantially reduces the policy’s sensitivity to a particular tokenization of the same action sequence. These results support stochastic action tokenization as an effective approach for learning autoregressive robot policies from limited demonstrations.

## VIII. ACKNOWLEDGMENT

This work was supported by the JST CREST project Real-World Intelligent Systems, “MORAL: Morphoception-Oriented Reasoning and Action with Language” (JP-MJCR2553) and FRONTia, a Japanese national program led by the Ministry of Economy, Trade and Industry of Japan (METI) and the New Energy and Industrial Technology Development Organization (NEDO), aimed at developing a

domestic multimodal foundation model for AI robots and physical AI. This work is also supported by JSPS KAKENHI Grant Number 25K21274. We thank Koshi Makihara for their support with the experimental setup.

## REFERENCES

- [1] T. Osa *et al.*, “An algorithmic perspective on imitation learning,” *Foundations and Trends in Robotics*, vol. 7, no. 1-2, pp. 1–179, 03 2018.
- [2] B. Zitkovich *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *CoRL*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 2165–2183.
- [3] S. Reed *et al.*, “A generalist agent,” *Transactions on Machine Learning Research*, 2022, featured Certification, Outstanding Certification.
- [4] D. Ghosh *et al.*, “Octo: An Open-Source Generalist Robot Policy,” in *RSS*, Delft, Netherlands, July 2024.
- [5] M. J. Kim *et al.*, “Openvla: An open-source vision-language-action model,” in *CoRL*, ser. Proceedings of Machine Learning Research, vol. 270, 2024, pp. 2679–2713.
- [6] K. Black *et al.*, “ $\pi_0$ : A Vision-Language-Action Flow Model for General Robot Control,” in *RSS*, 2025.
- [7] NVIDIA, “Gr00t n1: An open foundation model for generalist humanoid robots,” 2025.
- [8] S. Liu *et al.*, “Rdt-1b: a diffusion foundation model for bimanual manipulation,” in *ICLR*, vol. 2025, 2025, pp. 29 982–30 009.
- [9] J. Lee *et al.*, “Molmoact: Action reasoning models that can reason in space,” 2025.
- [10] K. Pertsch *et al.*, “FAST: Efficient Action Tokenization for Vision-Language-Action Models,” in *RSS*, 2025.
- [11] Z. Liang *et al.*, “Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies,” *arXiv preprint arXiv:2508.20072*, 2025.
- [12] J. Liu *et al.*, “HybridVLA: Collaborative diffusion and autoregression in a unified vision-language-action model,” in *ICLR*, 2026.
- [13] Y. Wang *et al.*, “Vq-vla: Improving vision-language-action models via scaling vector-quantized action tokenizers,” in *ICCV*, 2025, pp. 11 089–11 099.
- [14] R. Sennrich *et al.*, “Neural machine translation of rare words with subword units,” in *ACL*. Association for Computational Linguistics, 2016, pp. 1715–1725.
- [15] T. Kudo, “Subword regularization: Improving neural network translation models with multiple subword candidates,” in *ACL*. Association for Computational Linguistics, 2018, pp. 66–75.
- [16] I. Provilkov *et al.*, “BPE-dropout: Simple and effective subword regularization,” in *ACL*. Association for Computational Linguistics, 2020, pp. 1882–1892.
- [17] X. Wang *et al.*, “Multi-view subword regularization,” in *NAACL*. Association for Computational Linguistics, 2021, pp. 473–482.
- [18] A. Hurst *et al.*, “Gpt-4o system card,” *arXiv preprint arXiv:2410.21276*, 2024.
- [19] G. Comanici *et al.*, “Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities,” *arXiv preprint arXiv:2507.06261*, 2025.
- [20] L. Beyer *et al.*, “Paligemma: A versatile 3b vlm for transfer,” 2024.
- [21] S. Bai *et al.*, “Qwen3-vl technical report,” *arXiv preprint arXiv:2511.21631*, 2025.
- [22] M. Deitke *et al.*, “Molmo and pixmo: Open weights and open data for state-of-the-art vision-language models,” in *CVPR*, June 2025, pp. 91–104.
- [23] A. Brohan *et al.*, “RT-1: Robotics Transformer for Real-World Control at Scale,” in *RSS*, Daegu, Republic of Korea, July 2023.
- [24] K. Black *et al.*, “ $\pi_{0.5}$ : a vision-language-action model with open-world generalization,” in *CoRL*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 17–40.
- [25] G. R. Team, “Gemini robotics: Bringing ai into the physical world,” 2025.
- [26] H. Zhou *et al.*, “BEAST: Efficient tokenization of b-splines encoded action sequences for imitation learning,” in *NeurIPS*, 2025.
- [27] S. Lee *et al.*, “Behavior generation with latent actions,” in *ICML*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 26 991–27 008.
- [28] Y. Liu *et al.*, “FASTER: Toward powerful and efficient autoregressive vision-language-action models with learnable action tokenizer and block-wise decoding,” in *ICLR*, 2026.

- [29] M. Cognition *et al.*, “Distributional properties of subword regularization,” in *EMNLP*. Association for Computational Linguistics, 2024, pp. 10 753–10 763.
- [30] A. Sims *et al.*, “Stochastok: Improving fine-grained subword understanding in LLMs,” in *ICLR*, 2026.
- [31] M. Nagata, “A stochastic Japanese morphological analyzer using a forward-DP backward-A\* n-best search algorithm,” in *COLING*, Kyoto, Japan, Aug. 1994.
- [32] B. Liu *et al.*, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” in *NeurIPS*, vol. 36. Curran Associates, Inc., 2023, pp. 44 776–44 791.
- [33] P. Wu *et al.*, “Gello: A general, low-cost, and intuitive teleoperation framework for robot manipulators,” in *IROS*, 2024, pp. 12 156–12 163.
- [34] C. Chi *et al.*, “Universal Manipulation Interface: In-The-Wild Robot Teaching Without In-The-Wild Robots,” in *RSS*, Delft, Netherlands, July 2024.
- [35] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *ICLR*, 2019.
- [36] T. Kudo and J. Richardson, “SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” in *EMNLP*. Association for Computational Linguistics, 2018, pp. 66–71.
- [37] A. Khazatsky *et al.*, “DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset,” in *RSS*, Delft, Netherlands, July 2024.
- [38] C. Liu *et al.*, “Oat: Ordered action tokenization,” 2026.

## APPENDIX

### A. Hyperparameters for Constructing TOAST Tokenizer

Table V summarizes the hyperparameters of TOAST tokenizer construction used in Section V-B and Section V-C. In both LIBERO and real robot setups, the tokenizers adopt dimension-major flattening of quantized actions. The tokenizer construction corpus consists of 1M action chunks sampled from DROID, and end-effector poses and joint angles are used for LIBERO and DROID, respectively, to match the action representation the policy predicts.

### B. Hyperparameters for Policy Training

Table V shows the hyperparameters for training policies used in Section V-B and Section V-C. The policy parameters are tuned using AdamW [35]. Actions in continuous action chunks are represented as relative values to the current proprioceptive states [34]. In order to be robust to outliers, action values are normalized within the range  $[-1, 1]$  via quantile normalization, following prior work [10].

### C. Task Evaluation Rubric

In real-robot evaluation in Section V-C, we use a task evaluation rubric and judge if the rollout is successful or not. The following lists a rubric for each task.

- **Table Bussing** requires the robot to sort three objects on a table according to their type. Tableware items (e.g., plates and bowls) must be placed in the dish rack, while disposable items (e.g., paper cups) must be placed in the trash can. A rollout is considered successful if all objects are placed in their designated locations within 120 seconds; otherwise, it is considered a failure.
- **Grocery Bagging** requires the robot to place three objects into a paper bag. The manipulation targets are a bag of pasta, a bag of coffee, and a tube of chips. A rollout is considered successful if all objects are placed into the paper bag within 120 seconds without knocking it over; otherwise, it is considered a failure.

TABLE V: Hyperparameters for TOAST.

| Hyperparameter                 | LIBERO | Real Robot (FR3) |
|--------------------------------|--------|------------------|
| Vocabulary size                | 512    | 512              |
| $\alpha$ (smoothing parameter) | 0.1    | 0.5              |
| $n$ ( $n$ -best search)        | 64     | 64               |

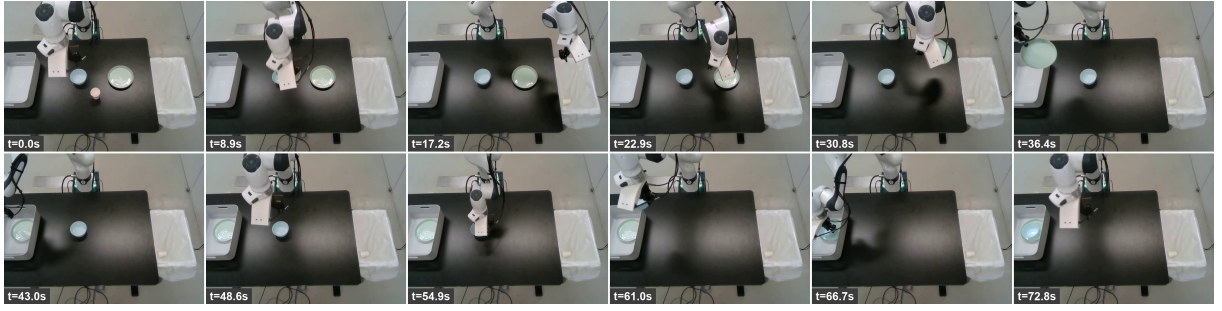
TABLE VI: Hyperparameters used for training policies.

| Hyperparameter | LIBERO               | Real Robot (FR3)     |
|----------------|----------------------|----------------------|
| Training steps | 30,000               | 30,000               |
| Warmup steps   | 1,000                | 1,000                |
| Batch size     | 32                   | 32                   |
| Learning rate  | $2.5 \times 10^{-5}$ | $2.5 \times 10^{-5}$ |
| Horizon        | 10                   | 20                   |

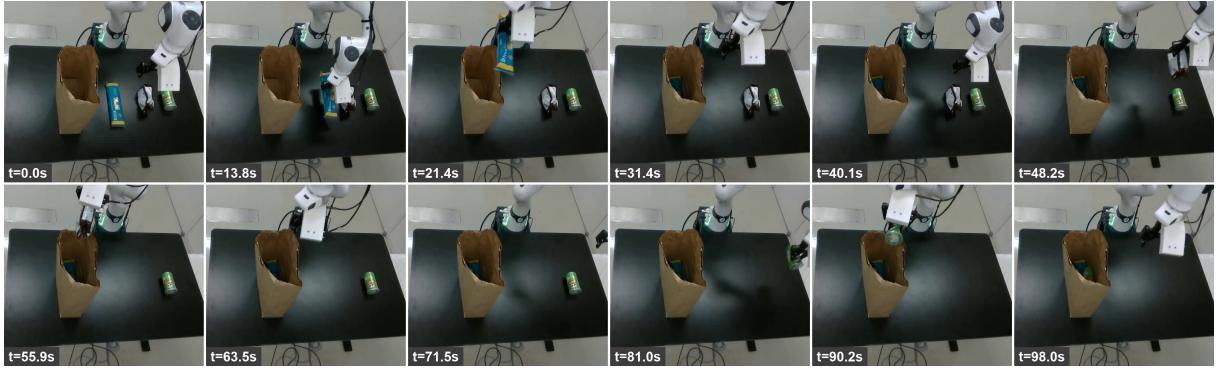
- **Breakfast Setup** requires the robot to place a coffee cup onto a coaster and transfer a slice of toast from a toaster onto a plate. A rollout is considered successful if both objects are placed in their designated locations within 120 seconds; otherwise, it is considered a failure. Additionally, if the robot lifts the toaster together with the toast, we stop the robot for safety reasons and mark the rollout as a failure.
- **Drawer Stowing** requires the robot to open a drawer, move an object from the table into the drawer, and then close the drawer. The manipulation target is a bowl, a banana, or a plum. A rollout is considered successful if, within 120 seconds, the object is inside the drawer and the drawer is fully closed; otherwise, it is considered a failure.

### D. Successful Rollout Visualization on Real-Robot Tasks

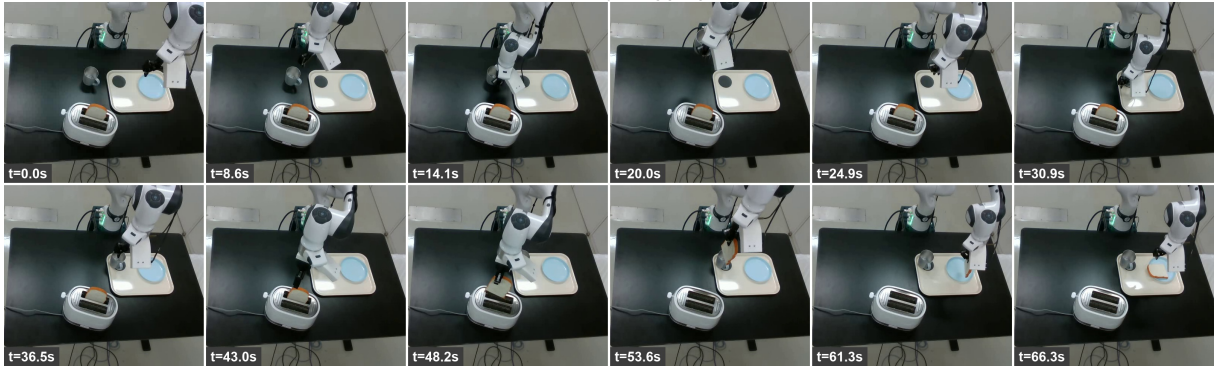
Figure 9 shows the keyframes from successful rollouts of policies trained with the proposed TOAST tokenizer. The first frame ( $t = 0.0s$ ) in each task corresponds to the initial state of the environment.



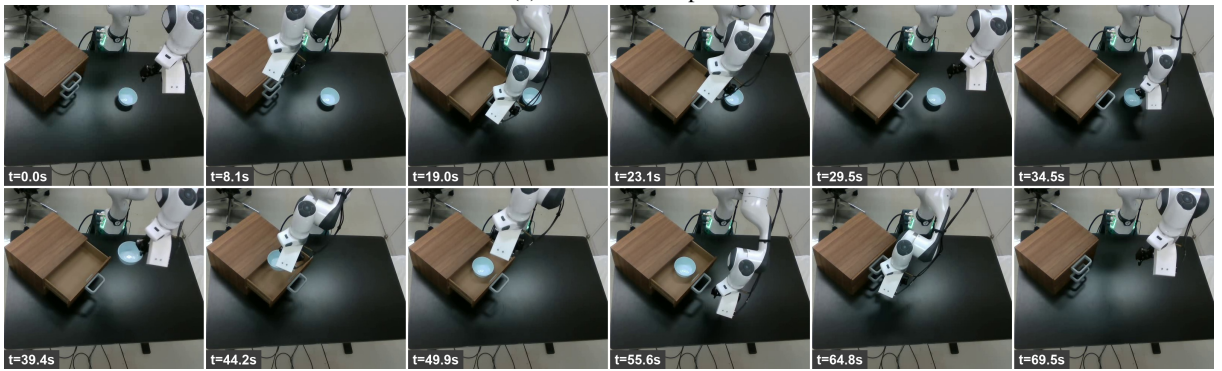
(a) Table Busing.



(b) Grocery Bagging.



(c) Breakfast Setup.



(d) Drawer Stowing.

Fig. 9: Successful rollouts on real-robot tasks.